

Jupyter notebook and starter code

Spring 2026

The Jupyter notebook setup and examples below are provided as an example of a setup that could be used for the CSE 291 class projects. You are not required to use these tools/setup - you are welcome to use any other tools/setup that you may be more familiar with and can thus use this document just as additional information illustrating one possible way to implement some of the analysis steps for the class projects.

Installing Jupyter notebook and packages

Jupyter notebooks provide a convenient way to process and visualize data as it can organize your source code, supports visualization and simplifies using Python packages to provide extensive additional functionality.

To set up Jupyter notebook, refer to [this link](#) and [the tutorial video](#). You can download our starter code from [here](#) (do let us know if you can not access).

In the starter code tutorial, we are going to use the following Python packages: [pandas](#), [seaborn](#), [numpy](#), [matplotlib](#) and [scipy](#), which should have been installed in your system if you selected the recommended Anaconda installation.

Classification problem of severe COVID-19 patients

Library import

```
%matplotlib inline
import pandas as pd
import seaborn as sns
import numpy as np
from matplotlib import pyplot as plt
from scipy import stats
```

Data loading and investigation

We are going to work on the data for [the COVID-19 Patient Sera](#) in our example. Specifically, we will analyze the data from [the peptide intensity matrix](#), where intensity values of 10,1461 peptide variants (each corresponds to a row) across 92 conditions/patients (with four main classes: Healthy, Severe, Non-severe, Symptomatic with non COVID) are reported.

To get started, we will download a tsv from the results view as described in the class projects handout: (i) Click the “Download” tab at the top of the page, (ii) Set “Include Entries” to the “All” option and (iii) Click the “Download” button to get a zip file containing the results table.

Once downloaded, we will load the tsv file into our workspace using a [pandas dataframe](#). Pandas dataframe is a data structure for two-dimensional and heterogeneous tabular data that supports efficient data manipulation operations and is thus suitable for the intensity matrix data.

Load the data file

```
INPUT_MAESTRO_DATA = "path/to/data"
variants = pd.read_csv(INPUT_MAESTRO_DATA, sep="\t", low_memory=False)
```

variants

	Algorithm	Filename	Cluster_index	Peptide	Unmodified_sequence	Charge	Intensity_for_cluster	Inte
0	.MODA.	specs_ms.mgf	960991	[304.207]GARLIPENDQIFTEVEMTTLE(K,304.207).V	.GARLIPENDQIFTEVEMTTLEK.	4	36.905893	
1	.MODA.	specs_ms.mgf	763982	I.[304.207]FTEVEMTTLE(K,304.207).V	.FTEVEMTTLEK.	3	11.686782	
2	.MSGFPLUS.	specs_ms.mgf	902201	K.[304.207]LYQPEYQEVSTEEQR.E	.LYQPEYQEVSTEEQR.	3	15.690234	
3	.MSGFPLUS.	specs_ms.mgf	935503	K.[304.207]AANSLEAFIFETQD(K,304.207).L	.AANSLEAFIFETQDK.	3	15.016824	
4	.MODA.	specs_ms.mgf	297961	R.[304.207]YSHDF(N,-56.985)FH.I	.YSHDFNFH.	3	33.768015	
...	...	...	...	...	...	...	...	...
101456	.MODA.	specs_ms.mgf	480358	K.[304.207]YLGE(E,-68.078)YV(K,304.207).A	.YLGEYVK.	3	15.671927	
101457	.MODA.	specs_ms.mgf	237950	K.[304.207]YL(G,55.921)EEYV(K,304.207).A	.YLGEYVK.	4	13.932377	
101458	.MODA.	specs_ms.mgf	1037953	K.{187.018}[304.207]YLGEYV(K,304.207).A	.YLGEYVK.	2	14.412538	
101459	.MODA.	specs_ms.mgf	1022758	[304.207]NTYE(K,361.237)YLGEYV(K,304.207).A	.NTYEKYLGEYVK.	3	13.872686	
101460	.MODA.	specs_ms.mgf	1021872	K.[304.207]YLGE(E,125.898)YV(K,304.207).A	.YLGEYVK.	2	14.889704	

101461 rows x 266 columns

The results table has many columns, however, the format and column headers for the most important data for the class projects are described in the “Data types” section on [the project handout](#).

By just typing the variable name and running the cell, we can see a summary of the data frame imported. You can also investigate the data by opening the tsv file using Excel/Google sheets or by using the online results view linked above and in the project description for the COVID-19 Patient Sera.

variants

	Algorithm	Filename	Cluster_index	Peptide	Unmodified_sequence	Charge	Intensity_for_cluster
0	.MODA.	specs_ms.mgf	960991	[304.207]GARLIPMDQIFTEVEMTTLE(K,304.207).V	.GARLIPMDQIFTEVEMTTLEK.	4	36.905893
1	.MODA.	specs_ms.mgf	763982	I.[304.207]FTEVEMTTLE(K,304.207).V	.FTEVEMTTLEK.	3	11.686782
2	.MSGFPLUS.	specs_ms.mgf	902201	K.[304.207]LYQPEYQEVSTEEQR.E	.LYQPEYQEVSTEEQR.	3	15.690234
3	.MSGFPLUS.	specs_ms.mgf	935503	K.[304.207]AANSLEAFIFETQD(K,304.207).L	.AANSLEAFIFETQDK.	3	15.016824
4	.MODA.	specs_ms.mgf	297961	R.[304.207]YSHDF(N,-56.985)FH.I	.YSHDFNFH.	3	33.768015
...	...	...	...	...	...	...	...
101456	.MODA.	specs_ms.mgf	480358	K.[304.207]YLGE(E,-68.078)YV(K,304.207).A	.YLGEYVK.	3	15.671927
101457	.MODA.	specs_ms.mgf	237950	K.[304.207]YL(G,55.921)EEYV(K,304.207).A	.YLGEYVK.	4	13.932377
101458	.MODA.	specs_ms.mgf	1037953	K.{187.018}[304.207]YLGEYV(K,304.207).A	.YLGEYVK.	2	14.412538
101459	.MODA.	specs_ms.mgf	1022758	[304.207]NTYE(K,361.237)YLGEYV(K,304.207).A	.NTYEKYLGEYVK.	3	13.872686
101460	.MODA.	specs_ms.mgf	1021872	K.[304.207]YLGE(E,125.898)YV(K,304.207).A	.YLGEYVK.	2	14.889704

The following lists all columns of the dataframe.

```
print("The number of column =", len(variants.columns))
for c in variants.columns:
    print(c)

The number of column = 266
Algorithm
Filename
Cluster_index
Peptide
Unmodified_sequence
Charge
Intensity_for_cluster
Intensity_for_unmodified_sequence
Num_G1_spectra_for_unmodified_sequence
Num_G2_spectra_for_unmodified_sequence
Num_G3_spectra_for_unmodified_sequence
Num_G4_spectra_for_unmodified_sequence
Num_G5_spectra_for_unmodified_sequence
Num_G6_spectra_for_unmodified_sequence
Empty.Empty.Empty..Empty.1_intensity_for_unmodified_sequence
Healthy.HC1.Healthy..HC1.1_intensity_for_unmodified_sequence
Healthy.HC10.Healthy..HC10.1_intensity_for_unmodified_sequence
Healthy.HC12.Healthy..HC12.1_intensity_for_unmodified_sequence
Healthy.HC13.Healthy..HC13.1_intensity_for_unmodified_sequence
```

Data extraction and processing

In the COVID-19 Patient Sera data, columns with the suffix “*_intensity_for_peptide_variant*” report the intensity values for identified peptide variants. For each of these columns, the first two elements of the prefix will tell you which measurement is reported in the column, like “**Healthy.HC10**.Healthy..HC10.1_intensity_for_peptide_variant” is the intensity column for a **Healthy** patient whose ID is “**HC10**”.

For the following analysis of peptide variants intensities we will just keep these intensity measurements, as well as the “Peptide” column to keep track of which measurement was for each peptide.

Once we have the columns necessary, there are just a few more steps to getting the data ready for analysis. The first step is to replace 0-intensity values as N/As (unknown) as they are not meaningful values in mass spectrometry (MS) measurement and might confuse later analysis.

The next steps are to transpose the matrix, so that each column is intensity for a given peptide, a step necessary for the many of the plots we will do. The first step to doing the transpose is to make the “Peptide” column the index of the dataframe, ensuring that this column will end up as the header for our transposed matrix. Finally, we just have to use the “.T” operation to transpose the matrix.

With the matrix transposed, the final step to getting the data ready is to add a new column for just the condition of the patient and then filter our data for just “Severe COVID-19” and “Non severe COVID-19” diagnoses.

Preprocessing

```
# we only need the intensities for each peptide, noted as "...intensity_for_peptide_variant"
variants_processed = variants[
    ['Peptide'] + [c for c in variants.columns if 'intensity_for_peptide_variant' in c]
]

# replace zero values (which are all intensities) in the processed matrix with N/As
variants_processed.replace(0.0, np.nan, inplace=True)

# set the index to be each Peptide
variants_processed = variants_processed.set_index('Peptide')

# transpose the matrix, so each column is a peptide
variants_processed = variants_processed.T

# only retain first two elements in the headers
variants_processed.index = variants_processed.index.map(lambda x: '.'.join(x.split('.')[0:2]))

# set the condition to the first element
variants_processed['Condition'] = variants_processed.index.map(lambda x: x.split('.')[0])

# only keep two conditions, for both severe and non-severe COVID-19
variants_processed = variants_processed[(variants_processed.Condition == "Severe-COVID-19")
    | (variants_processed.Condition == "Non-severe-COVID-19")]

variants_processed # print out to investigate
```

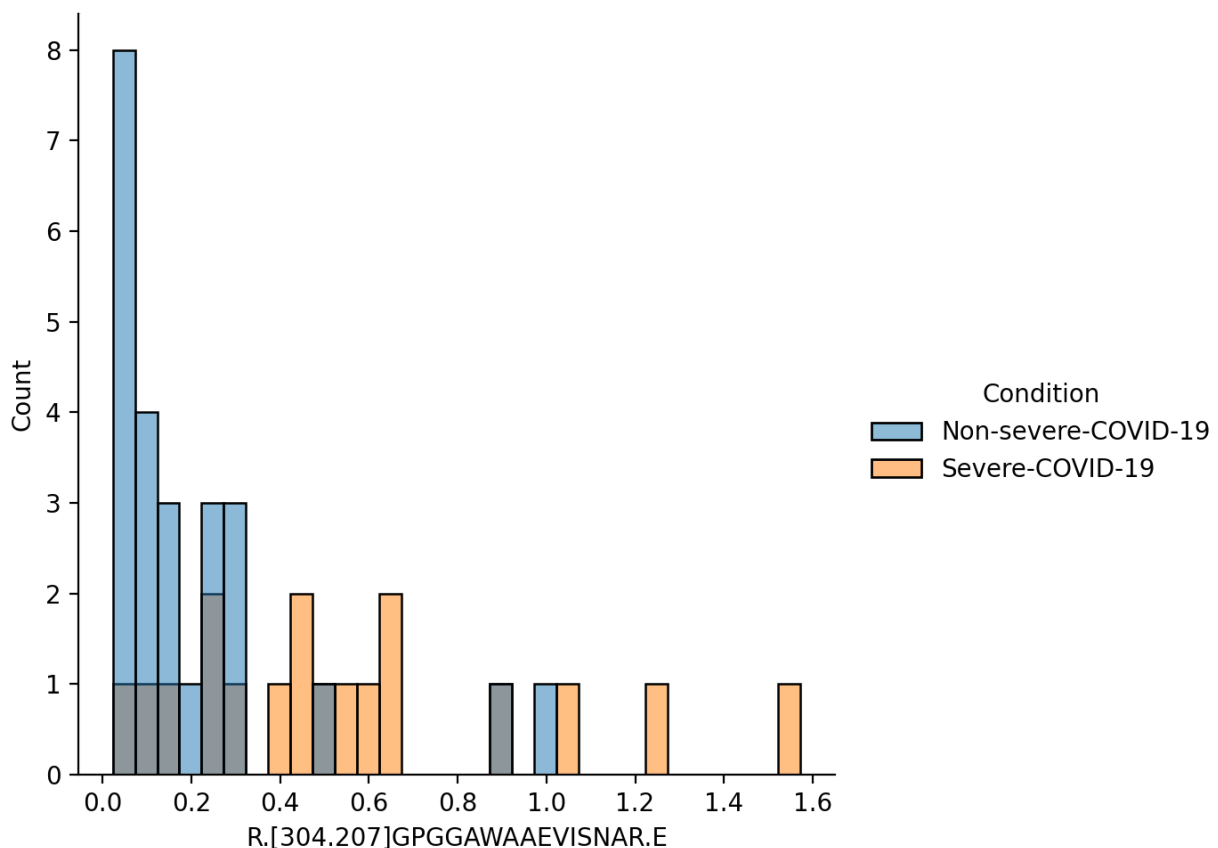
Data visualization

Data visualization is an important way to evaluate how the data distributions relate to the labels, to inspect your model's selections and to understand the limits of your model's results.

In the following code, we use [seaborn](#) and [matplotlib](#) to create histograms and plots. The following figure shows a histogram of intensities for a peptide variant "R.[304.207]GPGGAWAAEVISNAR.E" on severe and non-severe conditions. As you can see, although the means of intensities from the two classes are different (see statistical test below, p-value 0.003), the two classes still have substantial overlap. This shows that even though this peptide does change in a very significant way between severe and non-severe COVID19 patients, it alone is not sufficient to fully explain the difference between the two groups of patients.

Draw histograms/plots

```
his_data = sns.displot(variants_processed, x='R.[304.207]GPGGAWAAEVISNAR.E', hue='Condition', binwidth=0.05)
```



Since the previous histogram shows that one peptide is not sufficient to separate the two groups, it is also useful to create a 2D-scatter plot in seaborn to compare the intensity

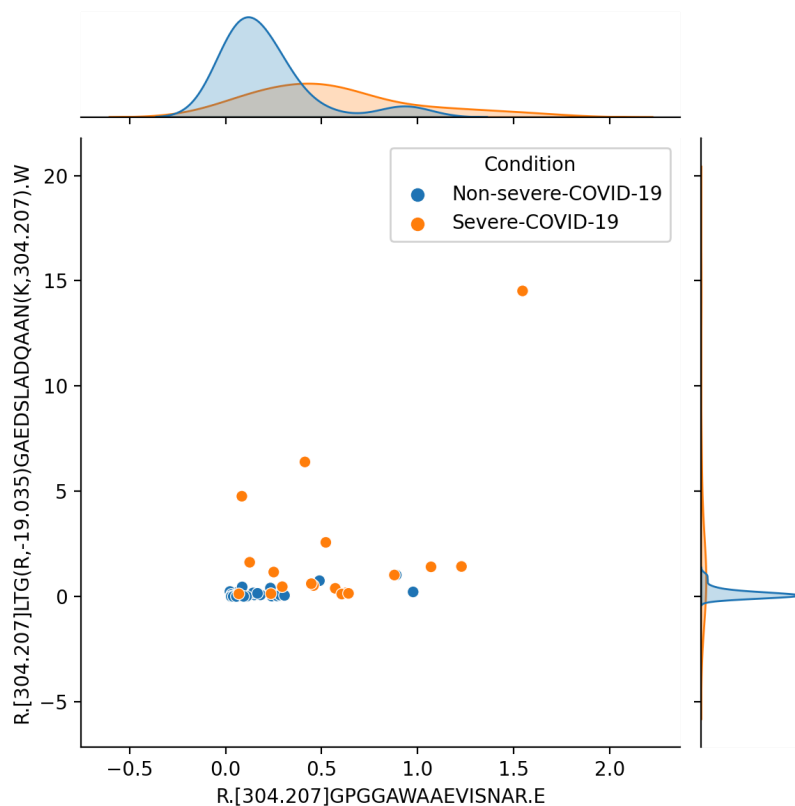
distributions for peptide variant pairs. The following plot illustrates this using the two peptides from protein sp|P0DJ19|SAA2_HUMAN with the most significant variation between the two groups of patients. While neither peptide provides a complete separation between the two groups of patients, it is clear that the second peptide (y axis) helps separate Severe COVID19 samples from the Non-Severe COVID19 samples in the range of values between 0 and 0.5 where the measurements from both groups overlap for the first peptide (x axis).

2D plot

```
#use the same filtered matrix from above
```

```
sns.jointplot(x=variants_processed["R.[304.207]GPGGAWAAEVISNAR.E"],
              y=variants_processed["R.[304.207]LTG(R,-19.035)GAEDSLADQAAN(K,304.207).W"],
              hue=variants_processed['Condition'])
```

```
<seaborn.axisgrid.JointGrid at 0x126feaf40>
```



Another reason why it's important to visualize your data is that missing values are unfortunately a very common occurrence in MS data. As such, your models may detect very significant variation of a peptide across group labels without revealing that the peptide may actually not have values for a substantial subset of the patients. In the following, we are going to use seaborn to draw a heatmap in order to see intensity distributions for the 12 peptide variants from sp|P0DJ19|SAA2_HUMAN whose variation was most significant across severe and non-severe COVID19 samples.

Some additional preprocessing steps are generally recommended before visualization of heat maps. First, since different peptides have different ranges of absolute intensity values measured across the various samples, it is recommended that the values of each peptide be normalized to z-scores across all samples where each peptide is observed - this centers the observations of each peptide on its mean value and displays its variation in the context of each peptide's standard deviation across all samples. Second, the z-scores are truncated to the range between -3 and 3 to avoid having the heatmap palette being set by the values of outliers with exceedingly high/low values away from the mean. Finally, we facilitate the visualization by transposing the matrix, so that each row in the heat map is a peptide and each column is a condition.

Data preprocessing for heat maps

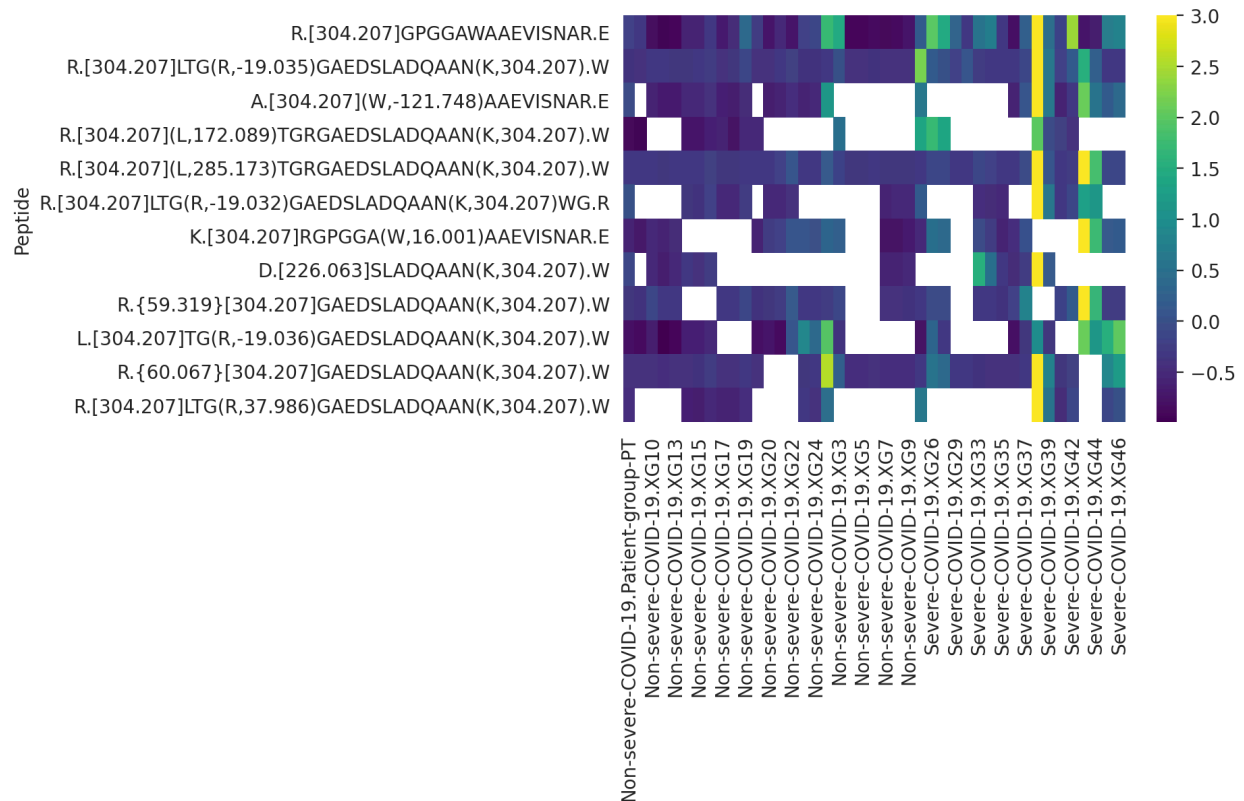
```
def z_score(df):  
    # copy the dataframe  
    df_std = df.copy()  
    # apply the z-score method  
    for column in df_std.columns:  
        df_std[column] = (df_std[column] - df_std[column].mean()) / df_std[column].std()  
        df_std[column] = df_std[column].map(lambda x: x if np.isnan(x) else min(3,max(-3,x)))  
    return df_std
```

```
#make a copy of the matrix for further processing in the heatmap  
variants_processed_for_heatmap = variants_processed.copy()  
# the top 12 variants for SAA2 in COVID19  
top_variants = ["R.[304.207]GPGGAWAAEVISNAR.E", "R.[304.207]LTG(R,-19.035)GAEDSLADQAAN(K,304.207).W", "A.[304.207](  
variants_processed_for_heatmap = variants_processed_for_heatmap.loc[:,list(top_variants)]  
# z-score transform the peptide intensities  
variants_processed_for_heatmap = z_score(variants_processed_for_heatmap)  
#transpose the matrix, so we can have peptides as rows  
variants_processed_for_heatmap = variants_processed_for_heatmap.T  
variants_processed_for_heatmap
```

With missing values shown in white, the heatmap clearly shows that various variants have a significant number of missing values, and are thus not sufficient (on their own) to explain the variation between the two groups of samples. It is thus important to be aware of this issue and to properly represent (in the input) and inspect (in the output) missing values when constructing models for analysis of MS datasets.

Draw the heat map for the top 12 variants

```
cmap = sns.color_palette("viridis", as_cmap=True)
with sns.axes_style("white"):
    sns.heatmap(variants_processed_for_heatmap, cmap=cmap)
```



Statistical analysis

We highly recommend [scipy.stats](#) as a package for your statistical analysis as it supports all common statistical functions with excellent guidelines and examples. In the code below we use two functions in [scipy.stats](#) to compute [the t-test](#) (standard/typical test for difference of means, makes Gaussian assumptions) and [the Mann-Whitney rank test](#) (non-parametric version of the test for difference of medians, does not make Gaussian assumptions) between two groups of intensities from a variant. These tests are to answer whether the means (t-test) or medians (Mann-Whitney) of the two groups are significantly different.

Statistical tests

```
severe_intensities = variants_processed["R.[304.207]GPGGAWAAEVISNAR.E"][variants_processed.Condition == "Severe-COV"]
non_severe_intensities = variants_processed["R.[304.207]GPGGAWAAEVISNAR.E"][variants_processed.Condition == "Non-se"]
print("T-test:", stats.ttest_ind(severe_intensities, non_severe_intensities, equal_var = False))
print("Mann-Whitney rank test:", stats.mannwhitneyu(severe_intensities, non_severe_intensities, alternative = 'two-
```

```
T-test: Ttest_indResult(statistic=3.241728321779687, pvalue=0.0032496041657580827)
Mann-Whitney rank test: MannwhitneyuResult(statistic=363.0, pvalue=0.0007117410518086113)
```


Note that before using a statistical function from any package you should learn about its parameters to make sure you will use the desired options. For example, you should set parameter **equal_var = False** to calculate the t-test without the default assumption from the function that the two lists tested have identical variances as the assumption is not necessarily true on the data. You should also set parameter **alternative = 'two-sided'** for the Mann-Whitney rank test as you want to test the difference between the two medians regardless of the direction (i.e., whether one is higher or lower than the other). The one-sided option should be used only if you are interested to test which specific median is only-higher or only-lower than the other.

Starter code for spectrum/spectrum matching (similar to peptide/spectrum matching)

In addition to using python packages mentioned at the beginning of this document ([pandas](#), [seaborn](#), [numpy](#), [matplotlib](#) and [scipy](#)), you would need to install [TensorFlow](#) or [PyTorch](#) for building the model in the starter code example. We provide both TensorFlow and PyTorch notebooks that you can choose by your preference. Whereas the TensorFlow notebook uses the [Keras](#) API to simplify model implementation details, The PyTorch one is more explicit in showing the details of the models. Both starter code options implement the same learning model and the resulting prediction performance is very similar. Links to the notebooks: [TensorFlow](#), [PyTorch](#).

Setup and import python packages

Firstly, you would need to Import python packages needed for the starter code. If using TensorFlow:

Set up TensorFlow and other packages

Import TensorFlow, numpy and sklearn into your program to get started:

```
In [86]: import tensorflow as tf
print("TensorFlow version:", tf.__version__)
import numpy as np
from sklearn.model_selection import train_test_split
from scipy.spatial.distance import cosine
import csv
import pandas as pd
```

TensorFlow version: 2.8.0

If you are following along in your own development environment, rather than [Colab](#), see the [install guide](#) for setting up TensorFlow for development.

Note: Make sure you have upgraded to the latest `pip` to install the TensorFlow 2 package if you are using your own development environment. See the [install guide](#) for details.

If you prefer PyTorch:

Set up PyTorch and other packages

Import PyTorch, numpy and sklearn into your program to get started:

```
In [27]: from argparse import Namespace
import torch
from torch import nn, optim
from torch.utils.data import Dataset, DataLoader
print("PyTorch version:", torch.__version__)
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, recall_score, precision_score, f1_score, confusion_matrix
from scipy.spatial.distance import cosine
from tqdm import tqdm
import csv
```

PyTorch version: 1.11.0

See the [install guide](#) for setting up PyTorch for development.

Note: Make sure you have upgraded to the latest `pip` to install the PyTorch package if you are using your own development environment.

Load the spectrum data and then process and transform the data

Next you want to load the mgf file that contains the list of spectra, which will be further used as data for your machine learning model. Each spectrum in a mgf will be specified by a unique scan ID, charge state and peptide mass, and followed by a list of m/z value-intensity pairs. To learn more about mgf format, please refer to [this link](#). The code snippet reads the mgf file, and

stores the m/z and intensity values of all the spectra in a python dictionary named *specs*, where the scan IDs are used as the keys.

Load a dataset

Load and prepare the dataset. The product of this stage is the set of intensity vectors and index pairs for training and testing the neural network.

```
[107]: from pyteomics import mgf
from collections import defaultdict

input_mgf = "./specs_ms.mgf"

specs = {}
with open(input_mgf) as input_mgf_file:
    with mgf.read(input_mgf_file, convert_arrays=0) as reader:
        for i, s in enumerate(reader):
            scan_number = int(s['params']['scans'])
            discretized_peaks = defaultdict(float)
            for mz, intensity in zip(s['m/z array'], s['intensity array']):
                discretized_peaks[round(mz*0.995)] += intensity
            specs[scan_number] = list(map(list, zip(*(discretized_peaks.items()))))

#you can print out a sample of the spectral data
#print(specs[3])
```

Next, from the data loaded you can construct a dataset for your further learning model. The code in the following builds sets X, Y of vectors of data and labels, respectively. The label information is from the file *output_pairs.tsv*, where each line shows a sample of spectrum pairs and the indication (True/False) that shows if the two spectra are from the same ID or not. For each of these pairs, you will construct two vectors of 2000 dimensions each representing an intensity vector from one of these spectra. Each of the dimensions corresponds to a 1 Da m/z bin, where peak intensities in the bin are summed up. Then the two vectors of 2000 dimensions are concatenated into a single vector of 4000 dimensions and add it to X. The label corresponding to the spectrum pair will be added to Y (either 1 or 0, with 1 if they are from the same peptide ID). The sets X, Y of data and labels will be used for the train/validation/test further.

Construct the dataset X, Y for the neural network later. This dataset will be further divided by train, validation, and test sets. X will contain concatenated vectors from pairs of spectral intensity vectors of 2,000 dimensions. Thus, each vector in X will have 4,000 dimensions. Y will contain 0/1 labels, where 1 indicates that the two corresponding vectors are from spectra of the same peptide, while 0 indicates they are not.

```
In [8]: X = []
Y = []
pair_idx = []
with open("/Users/db/Downloads/output_pairs.tsv") as fi:
    reader = csv.DictReader(fi, delimiter = '\t')
    for row in reader:
        idx1 = int(row['Index1'])
        idx2 = int(row['Index2'])
        if (idx1 in specs) and (idx2 in specs):
            (mz_arr1, intensity_arr1) = specs[idx1]
            (mz_arr2, intensity_arr2) = specs[idx2]
            vec1 = [0] * 2000
            vec2 = [0] * 2000
            for i in range(len(mz_arr1)):
                if mz_arr1[i] < 2000:
                    vec1[mz_arr1[i]] = intensity_arr1[i]
            for i in range(len(mz_arr2)):
                if mz_arr2[i] < 2000:
                    vec2[mz_arr2[i]] = intensity_arr2[i]
            max_intensity1 = float(max(intensity_arr1))
            max_intensity2 = float(max(intensity_arr2))
            vec1 = [x / max_intensity1 for x in vec1]
            vec2 = [x / max_intensity2 for x in vec2]

            X.append(vec1 + vec2)
            if row['Match'].upper() == 'TRUE':
                Y.append(1)
            elif row['Match'].upper() == 'FALSE':
                Y.append(0)
            else:
                print('WRONG FORMAT')
                break
            pair_idx.append((idx1, idx2))

# again, it is always good to print out sample data - to see if they are in your desired format/content
print(X[:2])
print(Y[:2])
print(X[-2:])
print(Y[-2:])
```

Next, you want to transform the data into numpy arrays and split them into train, test and validation sets using the [train_test_split](#) method from sklearn. In the example code, we want 30% of the data as the test. In 70% data remaining, we will further use a 20%/80% split for the validation and training respectively. When the split method is performed the data will be randomly shuffled. So you would like to keep track of the original indices from the *output_pairs.tsv* file of the data points. Note that the splitting percentages are just our recommendation on this task and dataset. You should decide the size of data splits based on your task and the availability of your data.

```
In [5]: Y = np.asarray(Y)
X = np.asarray(X).reshape(len(X), 4000, 1)
ori_idx = [i for i in range(len(X))] # we want to keep track the original indices of the data before splitting

# we are splitting 30% of the data for the test set, in the rest 20% will go for the validation set
x_train, x_test, y_train, y_test, idx_train, idx_test = train_test_split(X, Y, ori_idx, test_size=0.3, random_state=42)
x_train, x_val, y_train, y_val, idx_train, idx_val = train_test_split(x_train, y_train, idx_train, test_size=0.2, random_state=42)
print("N_train, N_val, N_test =", len(x_train), len(x_val), len(x_test))
```

Build a neural network for the prediction task

To build the neural network, we provide two options - either using TensorFlow or PyTorch. You can choose the one that you are most comfortable with.

Option 1: A neural network using TensorFlow

We use [the Keras APIs](#) to build a sequential model by linearly stacking 4 layers which include the first layer for the input data, the dense layer with output of 128 dimensions, the dropout layer for reducing overfitting and the last layer of two-dimension output for the prediction of two labels. The activation function used in the dense layer is also specified ([ReLU](#) in this case). Note that it

is a very simple network as an example. You can come up with your own neural network that is best fitted for your project problem and data.

Build a machine learning model

Build a `tf.keras.Sequential` model by stacking layers.

```
In [15]: model = tf.keras.models.Sequential([
          tf.keras.layers.Flatten(input_shape=(4000, 1)),
          tf.keras.layers.Dense(128, activation='relu'),
          tf.keras.layers.Dropout(0.2),
          tf.keras.layers.Dense(2)
        ])
```

In the following code, you will define the loss function, optimizer and evaluation metrics for the training process.

Define a loss function for training using `losses.SparseCategoricalCrossentropy`, which takes a vector of logits and a `True` index and returns a scalar loss for each example.

```
In [39]: loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
```

This loss is equal to the negative log probability of the true class: The loss is zero if the model is sure of the correct class.

This untrained model gives probabilities close to random (1/2 for each class), so the initial loss should be close to `-tf.math.log(1/2) ≈ 0.69`.

```
In [40]: loss_fn(y_train[:1], predictions).numpy()
```

```
Out[40]: 0.67064095
```

Before you start training, configure and compile the model using `Keras Model.compile`. Set the `optimizer` class to `adam`, set the `loss` to the `loss_fn` function you defined earlier, and specify a metric to be evaluated for the model by setting the `metrics` parameter to `accuracy`.

```
In [41]: model.compile(optimizer='adam',
                      loss=loss_fn,
                      metrics=['accuracy'])
```

Now, you are ready for the training process. Firstly, you might want to have a model checkpoint file (*weights.h5* in the code snippet) where you store the best model (in terms of the accuracy on *the validation set*) found so far during the training. This configuration of storing the best model can be specified by setting `save_best_only=True` as in the following code. The training process will be performed by calling `model.fit()`. In the example, it will try to minimize the loss function on the training data through 20 epochs. Finally, you would load the best model that has been found from all the epochs - from the checkpoint file.

Train and evaluate your model

Use the `model.fit` method to adjust your model parameters and minimize the loss:

```
In [59]: # Configure to save the best model with the highest accuracy on the VALIDATION set
checkpoint_filepath = './weights.h5'
model_checkpoint_callback = tf.keras.callbacks.ModelCheckpoint(
    filepath=checkpoint_filepath,
    monitor='val_accuracy',
    mode='max',
    save_best_only=True)

# Start training
model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=20, callbacks=[model_checkpoint_callback])

# The model weights (that are considered the best) are loaded into the model.
model.load_weights(checkpoint_filepath)

Epoch 1/20
1513/1513 [=====] - 3s 2ms/step - loss: 0.0049 - accuracy: 0.9989 - val_loss: 1.6190 - val
_accuracy: 0.8402
Epoch 2/20
1513/1513 [=====] - 4s 2ms/step - loss: 0.0035 - accuracy: 0.9992 - val_loss: 1.5926 - val
_accuracy: 0.8433
```

After the training and the best model selection are done. You can quickly evaluate the performance of this model by using `model.evaluate()`. The following code snippet is showing that your model achieves 84.17% of accuracy on the test set.

```
The model.evaluate method checks the models performance, usually on a "Validation-set" or "Test-set".

In [67]: model.evaluate(x_test, y_test, verbose=2)
811/811 - 1s - loss: 1.6413 - accuracy: 0.8417 - 628ms/epoch - 775us/step
Out[67]: [1.6413496732711792, 0.8417418599128723]

The spectrum match classifier is now trained to ~84% accuracy on this dataset. To learn more, read the TensorFlow tutorials.
```

As this is a prediction task, you might be interested in prediction probabilities for spectrum pairs on each label (0 or 1). The following code snippets help to return the probabilities by using a softmax. Each row in the printed-out tensor contains two float numbers corresponding to the probability values of predicting 0 or 1, respectively.

```
If you want your model to return a probability, you can wrap the trained model, and attach the softmax to it:

In [76]: probability_model = tf.keras.Sequential([
        model,
        tf.keras.layers.Softmax()
    ])

In [77]: print(y_test[:15])
print(probability_model(x_test[:15]))

[1 1 1 1 1 1 0 1 1 0 1 0 1 0]
tf.Tensor(
[[1.23026845e-17 1.00000000e+00]
 [0.00000000e+00 1.00000000e+00]
 [2.07742527e-02 9.79225755e-01]
 [8.37050527e-02 9.16294992e-01]
 [6.30990653e-23 1.00000000e+00]
 [1.15210083e-21 1.00000000e+00]
 [1.72261018e-04 9.99827743e-01]
 [9.99998927e-01 1.05774234e-06]
 [1.38445301e-07 9.99999881e-01]
 [2.25383566e-15 1.00000000e+00]
 [9.99999523e-01 4.27582847e-07]
 [1.24888857e-05 9.99987483e-01]
 [2.20093752e-05 9.99977946e-01]
 [1.98604554e-01 8.01395416e-01]
 [9.99999642e-01 3.31174846e-07]], shape=(15, 2), dtype=float32)
```

Option 2: A neural network using PyTorch

In this section, we provide an implementation with PyTorch for the same neural network with 4 sequential layers. You would define the neural network by subclassing `nn.Module`, and initialize the neural network layers in `__init__`. Then you will implement the operations on input data (the network's forward propagation) in the `forward` method.

Build a machine learning model

Build a PyTorch neural network by defining layers.

```
In [68]: class MatchingNet(nn.Module):
def __init__(self, input_dim=4000):
    """
    Model constructor, we define our layers here
    """
    super().__init__()

    self.linear_1 = nn.Linear(input_dim, 128)
    self.dropout = nn.Dropout(0.2)
    self.linear_2 = nn.Linear(128, 1)

def forward(self, x):
    """
    Forward propagation
    """
    x = torch.relu(self.linear_1(x))
    x = self.dropout(x)
    x = self.linear_2(x)

    y = torch.sigmoid(x)

    return y
```

Next, you would like to have a configuration for your model that specifies the learning rate, the number of epochs, batch size and the number of dimensions of the input data. Some supplemental functions for the model are also implemented that include *predict_proba()* for returning prediction probabilities and *predict()* for interpreting the probabilities to the actual prediction labels, where the label (either 0 or 1) of the higher probability will be selected as the final prediction.

Train and evaluate your model

In this step, we define our hyper-parameters and other config values. Then, we write our training loop to train the model.

```
In [60]: # Config for our model
config = {
    "learning_rate": 1e-3,
    "epoch": 20,
    "batch_size": 64,
    "input_dim": 4000
}

config = Namespace(**config)

In [61]: def predict_proba(net, features, device="cpu"):
    """
    Compute probability of being matched
    """
    with torch.no_grad():
        net.eval()
        data = MatchingDataset(features, None, requires_grad=False)
        loader = DataLoader(data, batch_size=512, shuffle=False)

        net.to(device)

        outputs = []
        for features_tensor in loader:
            features_tensor.to(device)
            preds = net(features_tensor).numpy().reshape(-1)
            outputs += list(preds)
        outputs = np.array(outputs)

    return outputs

def predict(net, features, device="cpu"):
    """
    Compute prediction label for each features
    """
    outputs = predict_proba(net, features, device)
    outputs[outputs >= 0.5] = 1
    outputs[outputs < 0.5] = 0
    return outputs
```

You can also configure the optimizer, the optimization metrics and the device used for the training as in the following code. If as the default, you would perform the training in CPU. Then in each epoch, you will compute prediction values, calculate the loss, perform the back propagation, and update network parameters to minimize the loss function. In each iteration, you might want to calculate the accuracy on the validation set to select the best model so far (on this validation) and save it into a file. This is also similar to what you try on the TensorFlow implementation above.

Start training on your neural network

```
In [41]: matching_net = MatchingNet(input_dim=config.input_dim)

# Training process
device = "cpu"
criterion = nn.BCELoss()
optimizer = optim.Adam(params=matching_net.parameters(), lr=config.learning_rate)

training_data = MatchingDataset(x_train, y_train, requires_grad=True)
#training_data = x_train, y_train
training_loader = DataLoader(training_data, batch_size=config.batch_size, shuffle=True)

matching_net.to(device)

losses = []
best_val_accuracy = 0.0

print("Start training...\n")
for epoch in range(config.epoch):
    print("Training epoch:", epoch)
    avg_loss = 0.0
    for features, labels in tqdm(training_loader):
        # Ensure that model is in train mode (This is important for using Dropout)
        matching_net.train()

        # Move our data to correct device
        features.to(device)
        labels.to(device)

        # Reset gradient to zero (PyTorch will accumulate gradient if we don't do this!)
        optimizer.zero_grad()

        # Compute prediction values
        outputs = matching_net(features)
        # Compute BCE loss
        loss = criterion(outputs, labels)

        losses.append(loss.item())
        avg_loss += loss.item()

    # Compute gradient using backward propagation
    loss.backward()
    # Update parameters
    optimizer.step()
    val_accuracy = accuracy_score(y_val, predict(matching_net, x_val))

    print("Training loss:", avg_loss / len(training_loader))
    print("Validation accuracy:", val_accuracy)
    if val_accuracy > best_val_accuracy:
        best_val_accuracy = val_accuracy
        torch.save(matching_net.state_dict(), "./matching_net_best_weight.pth")
    print("Saving new best model to ./matching_net_best_weight.pth")
    print()
```

Start training...

Training epoch: 0

100%|██████████| 757/757 [00:09<00:00, 82.98it/s]

Training loss: 0.6118777136427881

Validation accuracy: 0.7015455822795272

Saving new best model to ./matching_net_best_weight.pth

Training epoch: 1

100%|██████████| 757/757 [00:06<00:00, 121.37it/s]

Training loss: 0.5287472269887811

Validation accuracy: 0.7355979833044053

Saving new best model to ./matching_net_best_weight.pth

Evaluation and investigation the prediction results

To have a complete view on the model performance, you can output the prediction scores on [some metrics](#) that are usually applied for prediction tasks such as precision, recall and F1-score. All the scores can be easily calculated using the sklearn.metrics package.

Output the prediction scores on typical metrics

Return the prediction values (0/1) from the probabilities

```
In [80]: probs = probability_model(x_test)
print(probs.shape)
y_pred = []
for (p0, p1) in probs:
    if p0 > p1:
        y_pred.append(0)
    else:
        y_pred.append(1)
```

(25926, 2)

Output the scores on some important metrics

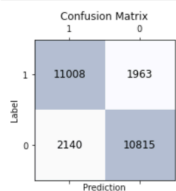
```
In [82]: from sklearn.metrics import precision_score, recall_score, confusion_matrix, classification_report, accuracy_score,
print('Accuracy:', accuracy_score(y_test, y_pred))
print('F1 score:', f1_score(y_test, y_pred))
print('Recall:', recall_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
```

Accuracy: 0.8417418807374836
F1 score: 0.8429112906313412
Recall: 0.8486624007401126
Precision: 0.8372376026772133

You can also output a confusion matrix for the binary prediction problem that visualizes the number of cases of correct/incorrect predictions in the test set on each label.

```
In [86]: import matplotlib.pyplot as plt

cmat = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(3, 3))
ax.matshow(cmat, cmap=plt.cm.Blues, alpha=0.3)
ax.invert_xaxis()
ax.invert_yaxis()
for i in range(cmat.shape[0]):
    for j in range(cmat.shape[1]):
        ax.text(x=j, y=i, s=cmat[i, j], va="center", ha="center", size="large")
plt.xlabel("Prediction")
plt.ylabel("Label")
plt.title('Confusion Matrix')
plt.show()
```



After you are done with the model implementation and the label prediction, it is important to check if the prediction works as your expectation of the model. We highly recommend that you do some manual evaluation on the results returned. For example, you can implement cosine similarity calculation for input intensity vectors as in the following code, and check if the prediction probabilities are correlated enough to the cosine similarities. This kind of checking can help you notice many bugs/mistakes from your model design and implementation.

Calculate the cosine similarities

```
In [26]: cosine_similarities = []
for x_concat in x_test:
    cosine_similarity = 1 - cosine(x_concat[:2000], x_concat[2000:]) # this cosine() from scipy.spatial.distance module
    cosine_similarities.append(cosine_similarity)

for i in range(15):
    print(cosine_similarities[i], idx_test[i])
```



```
0.842973672296923 17583
0.5138394729548531 38811
0.6135939643579007 17136
0.4406394978492685 4881
0.7141119126478495 14293
0.8530523421411509 31885
0.15073546289723094 32183
0.3476186030057289 54949
0.8458239489067457 5677
0.6677415902930671 2820
0.037162424663788274 54793
0.27800886549698 14813
0.11291547197308882 59654
0.8040333692264203 21271
0.37543219910325654 70100
```

On the prediction task, you can control the precision and sensitivity of your model by varying the prediction probability threshold (i.e. only return 1 if the probability of predicting label 1 is greater than a threshold). The following code allows you to calculate the precision and recall when applying different probability thresholds. This calculation is also applied for the cosine model, where you can set a cosine score threshold to decide returning 0 or 1 for the label prediction.

Precision and recall on different probability thresholds

```
In [64]: prob1 = probability_model(x_test).numpy()[0:,1]
prob_threshes = []
recalls = []
precisions = []
c_recalls = []
c_precisions = []

for i in range(101):
    prob_thresh = i / 100.0
    prob_threshes.append(prob_thresh)
    dy_pred = []
    for p1 in prob1:
        if p1 < prob_thresh:
            dy_pred.append(0)
        else:
            dy_pred.append(1)

    cy_pred = []
    for p1 in cosine_similarities:
        if p1 < prob_thresh:
            cy_pred.append(0)
        else:
            cy_pred.append(1)

    recall = recall_score(y_test, dy_pred)
    precision = precision_score(y_test, dy_pred)
    if (precision > 0.0) or (recall > 0.0):
        recalls.append(recall)
        precisions.append(precision)

    c_recall = recall_score(y_test, cy_pred)
    c_precision = precision_score(y_test, cy_pred)
    if (c_precision > 0.0) or (c_recall > 0.0):
        c_recalls.append(c_recall)
        c_precisions.append(c_precision)
    print(prob_thresh, precision, recall, c_precision, c_recall)
```



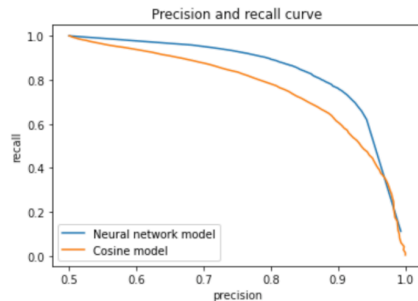
```
0.59 0.8598527293100688 0.8282322103153188 0.9019946961835582 0.6031146403515535
0.6 0.8605472197705207 0.826844499267597 0.9058338243959929 0.5925526173772261
```

With the precision and recall calculation, you can also plot precision and recall curves for the cosine and neural network models - by the following code. Each point in a curve would

represent a pair of precision (in x-axis) and recall (in y-axis) values at some probability threshold.

```
In [63]: import matplotlib.pyplot as plt

plt.plot(precisions, recalls, label='Neural network model')
plt.plot(c_precisions, c_recalls, label='Cosine model')
plt.title("Precision and recall curve")
plt.xlabel("precision")
plt.ylabel("recall")
plt.legend()
plt.show()
```



For the purpose of debugging or improving your model, it is important to look at your prediction results in relation to the actual labels. The following code outputs a pandas data frame that contains the spectrum pair information and your prediction results in the test set. By observing this kind of relation, you might notice which cases your model works very well or works badly, then can figure out the weakness of your learning model.

Investigation on the prediction results

Show the prediction results and information for the test set

```
In [48]: data_pairs = pd.read_csv("/Users/db/Downloads/output_pairs.tsv", sep="\t")
test_pairs = data_pairs.iloc[idx_test].copy()
test_pairs["Cosine Similarity"] = cosine_similarities
test_pairs["Prediction Probability"] = probability_model(x_test).numpy()[0,1]
test_pairs["Prediction Label"] = y_pred
test_pairs
```

Out[48]:

ex1	PM1	ID1	Index2	PM2	ID2	Match	Cosine Similarity	Prediction Probability	Prediction Label
499	2127.76	HHHTQIASTFIQVQAI/3	217596	2157.72	HHHTQIASTFIQVQAI/3	True	0.842974	0.999993	1
298	3270.54	AVLASMDNENMHTPDIGGQTTSEAIQDVIR/3	544301	3270.56	AVLASMDNENMHTPDIGGQTTSEAIQDVIR/3	True	0.513839	1.000000	1
602	1404.63	YTFAAHMDGTYK/2	210209	1405.63	YTFAAHMDGTYK/2	True	0.613594	0.593761	1
625	1543.84	LQAEAPHIVVGTGPR/3	46737	1544.86	LQAEAPHIVVGTGPR/3	True	0.440639	0.883056	1
811	1989.06	LY(C,57.021)IYVAIGQK/3	171072	2238.80	LY(C,57.021)IYVAIGQK/3	True	0.714112	0.999967	1
...	...	...	...	...	...	...	...	...	...
197	1588.89	THINIVVIGHVDSGK/3	57373	1589.87	THINIVVIGHVDSGK/3	True	0.186311	0.957144	1
443	1039.64	VKPLLQVSR/2	49490	1039.58	GPEVDIKGPK/2	False	0.658205	0.188981	0
675	2210.10	LGEHNIDVLEGNEQFINAAK/3	551962	3315.36	LGEHNIDVLEGNEQFINAAK/3	True	0.018541	0.998367	1
872	1770.93	IDIPSFDPWPIAPFPR/2	391887	1770.93	NIIFVETSFTLETR/2	False	0.081127	0.000440	0
784	1872.98	TTLPADPFALAPFSPK/2	437819	1873.02	SILPSNLLTV(C,57.021)QEGTLK/2	False	0.362578	0.275824	0

... x 10 columns

Congratulations! You have successfully built a basic neural network for predicting spectrum matches and evaluated its prediction performance on the real data.